ANN (Artificial Neural Networks) Practicals

Structured Notes

Non-Linear Function Approximation, Loss Functions,
Back-Propagation, Autograd, and Training Dynamics

Contents

1 Recap: PyTorch and nn.Linear **2**

2 Training an ANN to Approximate $y = x^2$ **2**
2.1 Why a Linear Model Cannot Fit $y = x^2$ . . . 2
2.2 Setting Up the Data . . . 2
2.3 Building and Training the Model . . . 2
2.4 Hyperparameter Tuning . . . 3
2.5 Why ReLU Enables Curve Approximation . . . 3
2.6 ANN Lecture 1 Summary . . . 3
2.7 Shape Analysis for the $y = x^2$ Model . . . 4

3 Loss Functions **4**
3.1 Mean Squared Error (MSE) . . . 4
3.2 Cross-Entropy Loss . . . 5
3.3 Loss as a Function of Network Parameters . . . 5

4 Back-Propagation **5**
4.1 The Chain Rule . . . 5
4.2 Forward Pass (Two-Layer ANN) . . . 6
4.3 Backward Pass: Computing Gradients . . . 6

5 Autograd and Gradient Inspection **7**
5.1 Inspecting Gradients with model.named_parameters() . . . 7
5.2 nn.Sequential vs nn.Module . . . 7

6 Understanding Gradient Magnitudes **7**

7 Training Dynamics **8**
7.1 The Three Components of Training . . . 8
7.2 The Role of Optimizers . . . 8
7.3 Gradient Descent Update Rule . . . 8

8 Role of Learning Rate **9**
8.1 Effect on Convergence . . . 9
8.2 Comparing Learning Rates . . . 9

1 Recap: PyTorch and nn.Linear

PyTorch uses `torch` tensors to accelerate computation on GPUs. The key point about the `nn.Linear` module is that when we pass `nn.Linear(D, H)`, it stores the weight matrix as shape `[H, D]` — the transpose of the input-output convention. This is because internally the forward method computes `X @ W.T + b`.

The `unsqueeze` function adds an extra dimension. For example, if a tensor has shape `[200]`, applying `unsqueeze(1)` converts it to shape `[200, 1]`, making it a column vector. This is important for matrix multiplication, which requires column vectors.

2 Training an ANN to Approximate $y = x^2$

2.1 Why a Linear Model Cannot Fit $y = x^2$

A purely linear model cannot represent the curvature of $y = x^2$ because:

$$\frac{d^2}{dx^2}(\text{linear function}) = 0$$

But for $y = x^2$:

$$\frac{d^2}{dx^2}(x^2) = 2 \neq 0$$

Therefore, a non-linear model is required. A hidden layer plus activation enables non-linear function approximation.

2.2 Setting Up the Data

Random data is generated between -2 and 2 with 200 data points. The target function is $y = x^2$. The input is reshaped to a column vector using `unsqueeze(1)` to get shape `[200, 1]`.

2.3 Building and Training the Model

A two-layer ANN with one input feature is constructed:

- Input dimension: 1
- Hidden layer: 16 neurons (initially)
- Output: 1
- Loss function: Mean Squared Error (MSE)
- Optimizer: SGD with learning rate 0.05

The training loop follows the standard pattern:

1. Reset gradients to zero (`optimizer.zero_grad()`)
2. Forward pass: compute predictions
3. Compute loss
4. Backward pass: compute gradients (`loss.backward()`)
5. Update weights (`optimizer.step()`)

Training for 500 epochs showed the loss decreasing strongly because x^2 is a simple function. However, this may lead to over-training. Training for 100 epochs showed good loss decrease, and the loss curve

converged smoothly around epoch 100.

2.4 Hyperparameter Tuning

Key Observation: With 16 hidden neurons, training from 100 to 300 epochs could not achieve a precise fit. The network was still under-fitted — it captured the curvature but could not approximate it smoothly.

Solution: Increasing the number of neurons from 16 to 32 resulted in much faster convergence and a better approximation. With 32 neurons, the model converged by epoch 150.

- With 16 neurons: training up to 300 epochs still under-fitted; the model hit its approximation limit.
- With 32 neurons: much faster convergence, the function was approximated precisely.
- The number of epochs determines **how long** the model trains, while the number of neurons determines **how well** it can approximate.
- Instead of increasing depth (more neurons in one layer), increasing breadth (adding another layer with fewer neurons) can also work.

How to Determine Epochs:

- Through the loss curve: when the loss flattens (converges to a stable point), further training is unnecessary.
- Using early stopping: if validation loss does not change for 10–20 epochs, stop training.
- The number of epochs does not depend on dataset size; it depends on how fast the model converges.

In deep learning, neuron counts typically follow powers of 2: 16, 32, 64, 128, 256. This is not a strict requirement but often works better in practice.

2.5 Why ReLU Enables Curve Approximation

ReLU creates a piecewise linear function.

Each hidden neuron introduces a **kink** — a change in slope. Because some neurons fire and some do not, the slope changes at different points.

By combining many such neurons, the network builds a **piecewise linear approximation** of a curved function.

- Without activation: the output is a single straight line (linear function). It can never approximate a curve.
- With ReLU: each neuron contributes a slope segment. The network fits the curve piece by piece.
- More neurons = more precise linear segments = better approximation.

Resource Efficiency: Increasing neurons from 32 to 64 would fit the curve but wastes resources. The goal is to find the minimum number of neurons that can adequately fit the function.

2.6 ANN Lecture 1 Summary

- A layer is a **matrix transformation**.

- Activation introduces **non-linearity**.
- Without activation, stacked layers collapse into one affine map.
- With activation, the network can approximate non-linear functions.
- Core flow: Matrix Transformation → Non-linearity → Feature Creation → Function Approximation.

This is the foundation of artificial neural networks.

2.7 Shape Analysis for the $y = x^2$ Model

For the two-layer ANN approximating $y = x^2$:

- Shape of Z_1: $[N, H]$ (number of samples × number of neurons)
- Shape of A_1: $[N, H]$ (same shape after activation)
- Shape of $\hat{Y}$: $[N, 1]$ (output dimension)

3 Loss Functions

In supervised learning, the network produces predictions $\hat{Y}$ based on input, weights, and bias. The **loss function** measures how far the prediction is from the true target Y.

The loss depends on:

- Prediction $\hat{Y}$ and target Y
- Network parameters: weights W and biases b

Mathematically: $L = L(W_1, b_1, W_2, b_2)$

Training a neural network means minimizing this loss.

3.1 Mean Squared Error (MSE)

For regression problems, MSE is commonly used:

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

In vector form:

$$\text{MSE} = \frac{1}{N}\|\hat{Y} - Y\|_2^2$$

Properties:

- Penalizes larger errors more heavily (quadratic penalty)
- Smooth and differentiable
- Suitable for regression tasks

Code demonstration:

```
y = torch.randn(5, 1)
predicted = torch.randn(5, 1)
mse = (predicted - y).pow(2).mean()
```

3.2 Cross-Entropy Loss

For classification problems, cross-entropy is used.

Binary Cross-Entropy (BCE):

$$\text{BCE} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)]$$

This is derived from the **logits** — the log of odds from maximum likelihood estimation.

Why Cross-Entropy?

- Measures the difference between the true distribution and the predicted distribution.
- Strongly penalizes confident wrong predictions.
- Works naturally with sigmoid or softmax outputs because their values are between 0 and 1.
- Sigmoid is for binary classification; softmax is the multi-class extension.
- Cross-entropy aligns with probability distribution and log likelihood.

Code demonstration:

```
y = torch.tensor([0.0, 1.0])        # True labels
predicted = torch.tensor([0.9, 0.6]) # Predicted probabilities
criterion = nn.BCELoss()
loss = criterion(predicted, y)
```

Tensor values should be in floating point because floating point numbers are continuous quantities, which enables differentiation.

3.3 Loss as a Function of Network Parameters

The predicted value $\hat{Y}$ is a function of W_1, b_1, W_2, b_2. Therefore:

$$L = L(W_1, b_1, W_2, b_2)$$

This is a function defined over a **high-dimensional space**. For example, if W_1 has 16×1 parameters and W_2 has 1×16 parameters, the total number of parameters exceeds 30.

The loss is a **surface** in this parameter space, and **training means moving downhill on this surface** to reach the minimum loss.

Key Chain:

- Prediction depends on parameters
- Loss depends on prediction
- Therefore, loss is directly dependent on parameters

The purpose of the loss function is **optimization** — to update the weights.

4 Back-Propagation

4.1 The Chain Rule

Back-propagation is nothing more than the **chain rule** from calculus.

If y is a function of u, and u is a function of x:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

In a neural network:

- Loss depends on output
- Output depends on hidden layers
- Hidden layers depend on parameters

Therefore, gradients must flow from **loss** $\rightarrow$ **output** $\rightarrow$ **hidden layers** $\rightarrow$ **parameters**.

This backward flow of gradients is called **back-propagation**.

4.2 Forward Pass (Two-Layer ANN)

The forward pass for a two-layer ANN:

1. $Z_1 = XW_1^T + b_1$ (first linear transformation)
2. $A_1 = \phi(Z_1)$ (activation function)
3. $Z_2 = A_1W_2^T + b_2$ (second linear transformation)
4. $\hat{Y} = Z_2$ (output)
5. $L = \text{loss}(\hat{Y}, Y)$ (compute loss)

We can only update weights and biases through differentiation. Features come from data and cannot be explicitly changed.

4.3 Backward Pass: Computing Gradients

Step 1: Gradient at Output

$$\frac{\partial L}{\partial \hat{Y}} = \frac{2}{N}(\hat{Y} - Y)$$

Step 2: Gradients for W_2 and b_2

Since $Z_2 = A_1W_2^T + b_2$:

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial Z_2} \cdot \frac{\partial Z_2}{\partial W_2} = \left(\frac{\partial L}{\partial Z_2}\right)^T A_1$$

$$\frac{\partial L}{\partial b_2} = \sum \frac{\partial L}{\partial Z_2}$$

Shape Reasoning: Z_2 has shape $[N, O]$, A_1 has shape $[N, H]$, so $\frac{\partial L}{\partial W_2}$ has shape $[O, H]$ — matching the shape of W_2. The transpose maintains shape symmetry.

Step 3: Gradient flowing to Hidden Layer

Loss affects hidden layer through output layer:

$$\frac{\partial L}{\partial A_1} = \frac{\partial L}{\partial Z_2} \cdot W_2$$

Apply the activation derivative:

$$\frac{\partial L}{\partial Z_1} = \frac{\partial L}{\partial A_1} \odot \phi'(Z_1)$$

where $\odot$ denotes element-wise (Hadamard) multiplication.

Step 4: Gradients for W_1 and b_1

$$\frac{\partial L}{\partial W_1} = \left(\frac{\partial L}{\partial Z_1}\right)^T X$$

Here, the features X appear because $Z_1 = XW_1^T + b_1$.

Why does $\frac{\partial L}{\partial W_2}$ involve A_1?

Because the output of the first layer, after going through the activation, becomes the input for the second layer. So:

$$Z_2 = W_2 \cdot A_1^T + b_2$$

Therefore $\frac{\partial L}{\partial W_2}$ naturally depends on A_1.

Similarly, $\frac{\partial L}{\partial W_1}$ involves X because $Z_1 = X \cdot W_1^T + b_1$.

5 Autograd and Gradient Inspection

5.1 Inspecting Gradients with model.named_parameters()

To see how gradients are being calculated or to inspect gradient values, use:

```
for name, param in model.named_parameters():
    print(name,
          param.shape,
          param.grad.shape,
          param.grad.abs().mean().item())
```

This prints for each layer: the parameter name, the parameter shape, the gradient shape, and the mean absolute gradient value.

The gradient calculation is done automatically by PyTorch's **autograd** system during `loss.backward()`. There is no need to write the back-propagation equations manually.

5.2 nn.Sequential vs nn.Module

nn.Module: Requires creating a full class with `__init__` and `forward` methods.

nn.Sequential: A simpler alternative where layers are passed directly without creating a class:

```
model = nn.Sequential(
    nn.Linear(1, 32),
    nn.ReLU(),
    nn.Linear(32, 1)
)
```

Inside `nn.Sequential`, pass the linear layers and activation functions in order. This avoids the need to define a class and is convenient for simple architectures.

6 Understanding Gradient Magnitudes

Large gradients: Big parameter updates, but risk of instability.

Small gradients: Very slow learning, possible vanishing gradient problem.

Gradient magnitude tells us how strongly each layer is responsible for correcting the error.

Example from gradient inspection:

- Second layer (output layer): bias gradient is relatively large (> 1).
- First layer (hidden layer): weight gradient is very small (≈ 0.04).

The first layer gradient is much smaller, which means it has a chance of falling into the **vanishing gradient problem**. The gradient values were computed after just the first backward pass (not after full training).

If activation derivative is near zero: Hidden layer will stop learning entirely — this is the vanishing gradient problem.

7 Training Dynamics

7.1 The Three Components of Training

Training a neural network involves three sequential steps:

1. **How the loss is calculated:** Forward pass produces predictions, then loss function measures error.
2. **How the gradients are calculated:** Back-propagation computes gradients using the chain rule.
3. **How the gradients update the weights:** Optimizers use the gradients to update parameters.

7.2 The Role of Optimizers

Critical Distinction:

- Optimizers are used to **update the weights**, NOT to calculate the gradient.
- Gradients are calculated through **back-propagation**.
- Once gradients are computed, optimizers use them to update the weights using the gradient values.

This is a common interview question: "What is the use of optimizers?" Answer: To update the weights using gradients.

7.3 Gradient Descent Update Rule

The fundamental update rule:

$$W \leftarrow W - \eta \cdot \frac{\partial L}{\partial W}$$

where η is the learning rate.

Interpretation:

- Gradient ($\frac{\partial L}{\partial W}$): provides the **direction** to move.

- Learning rate (η): determines the **step size**.

Training behavior depends critically on both components. Optimization is not just about the gradient — it is about **gradient** $\times$ **learning rate**.

8 Role of Learning Rate

8.1 Effect on Convergence

Small learning rate:

- Very small steps
- Slow convergence
- Stable but inefficient

Large learning rate:

- Large parameter jumps
- May overshoot the minimum
- Risk of divergence

8.2 Comparing Learning Rates

Experiment: Training ANN to approximate $y = x^2$ with different learning rates:

Learning Rate	Behavior	Convergence
0.001	Very small steps	Very slow, converges after 400+ epochs
0.05	Smooth updates	Good, smooth convergence
0.1	Fast updates	Fastest convergence, smooth
0.5	Large jumps	Overshoots initially, then stabilizes

At learning rate 0.5, the loss initially decreased, then overshot the minimum (loss went higher), but eventually stabilized as epochs progressed. The initial undershoot was caused by missing the minimum point.

At learning rate 0.001, convergence happens but very slowly — it may take 500 epochs to converge where 0.1 converges in 100 epochs.

Important Clarification: Learning rate and vanishing gradient have **no connection**. Learning rate is just the step size. A small learning rate (e.g., 0.001) means very small updates, but it will eventually converge as the number of epochs increases. Vanishing gradient is caused by activation function derivatives being near zero.

Key Insight: The smoother the convergence graph, the better the learning rate choice. Plotting the loss curve is essential for evaluating learning rate selection.

In the Adam optimizer, the learning rate still plays a role even though Adam also uses momentum (β) parameters. The effect of learning rate is present in all optimizers.